AMAZON DELIVERY

Agente Intelligente per il Supporto alle Decisioni di Consegna

GRUPPO DI LAVORO

- Michele Russo, *775735*, m.russo146@studenti.uniba.it

REPOSITORY GITHUB

- **URL :** https://github.com/MikeRvsso/ICON25_26.git

A.A. 2025/2026

Indice

1 Introduzione **3**
1.1 Dominio di interesse . . . 3
1.2 Knowledge Based System (KBS) . . . 3
1.3 Origine dei Dati . . . 3
1.4 Costruzione della Conoscenza (Preprocessing) . . . 4
1.4.1 Analisi delle Correlazioni . . . 4

2 Apprendimento Supervisionato **7**
2.1 Sommario . . . 7
2.2 Strumenti utilizzati . . . 7
2.3 Decisioni di Progetto . . . 7
2.3.1 Trattamento dei dati . . . 7
2.3.2 Selezione dell'Algoritmo . . . 7
2.3.3 Validazione della Robustezza (K-Fold Cross Validation) . . . 8
2.4 Valutazione . . . 9
2.4.1 Analisi dei Risultati (Random Forest) . . . 10

3 Ragionamento con Vincoli **12**
3.1 Sommario . . . 12
3.2 Strumenti utilizzati . . . 12
3.3 Decisioni di Progetto . . . 12
3.3.1 Trattamento dei Dati . . . 12
3.3.2 Modellazione del Problema (CSP) . . . 13
3.4 Scelta dell' Algoritmo . . . 13
3.4.1 Backtracking Search . . . 13
3.4.2 Simulated Annealing (SA) . . . 14
3.4.3 Iterative Best Improvement . . . 15
3.5 Valutazione e Risultati . . . 16
3.6 Conclusioni sul Motore Scelto . . . 16

4 Ricerca di Soluzioni in Spazi di Stati **18**
4.1 Sommario . . . 18
4.2 Strumenti utilizzati . . . 18
4.3 Scelta Algoritmi . . . 18
4.3.1 Algoritmo di Ricerca A* . . . 18
4.3.2 Greedy Best-First Search . . . 19
4.3.3 Lowest Cost First Search . . . 20
4.4 Valutazione . . . 21
4.4.1 Analisi dell'Efficienza (Nodi Espansi) . . . 22
4.4.2 Analisi della Qualità (Ottimalità del Percorso) . . . 22
4.4.3 Conclusione Algoritmo Scelto : A* . . . 22

5 Ragionamento Probabilistico (Bayesian Network) **23**
5.1 Sommario . . . 23
5.2 Strumenti Utilizzati . . . 23
5.3 Decisioni di Progetto . . . 23
5.3.1 Architettura del Grafo e Dipendenze Causali . . . 23

5.3.2 Discretizzazione delle Variabili 24
5.3.3 Strategia di Apprendimento e Gestione dell'Incertezza 25
5.4 Sviluppo del Modello: Apprendimento e Inferenza 25
5.4.1 Apprendimento delle Tabelle di Probabilità Condizionata (CPT) . . 25
5.4.2 Configurazione del Motore di Inferenza 25
5.5 Valutazione dei Risultati 26
5.5.1 Analisi degli Scenari 26
5.5.2 Diagnostica Inversa (Backward Reasoning) 26

6 Ragionamento Proposizionale (Prolog) 28
6.1 Sommario 28
6.2 Strumenti Utilizzati 28
6.3 Decisioni di Progetto 28
6.3.1 Campionamento dei dati per la verifica deduttiva 28
6.3.2 Definizione dei criteri di merito 29
6.3.3 Categorizzazione delle priorità e mitigazione del rischio 29
6.3.4 Architettura modulare e scalabilità 29
6.4 Modellazione Logica 29
6.4.1 Rappresentazione dei Fatti (Knowledge Base) 29
6.4.2 Logica delle Regole di Business 30
6.4.3 Regola del Bonus Merito 30
6.4.4 Regola di Alert Logistico (Notifica Ritardo) 30
6.4.5 Regola di Gestione Priorità 30
6.4.6 Implementazione delle Regole Decisionali 30
6.5 Valutazione 31
6.5.1 Valutazione Bonus Merito (Agenti) 31
6.5.2 Valutazione Alert e Priorità (Ordini) 31
6.6 Conclusioni sul Modulo Logico 32

7 Conclusioni e Sviluppi Futuri 33
7.1 Integrazione e Sinergia tra i Moduli 33
7.2 Utilizzo di Flussi di Dati in Tempo Reale 33
7.3 Chiusura del Progetto 33

Riferimenti Bibliografici 33

1 Introduzione

Il software di supporto ha come obiettivo di sviluppare un Assistente Intelligente per la Logistica che aiuti a ottimizzare l'intero processo di consegna giornaliero. Il sistema agisce come un coordinatore capace di analizzare il contesto operativo per prevedere i tempi di percorrenza , assegnare il carico al veicolo e calcola il percorso ottimale su mappa. Infine, attraverso un motore probabilistico, valuta i rischi di mancata consegna, offrendo un supporto decisionale completo.

1.1 Dominio di interesse

Il progetto si colloca nel contesto della **logistica last-mile** (consegna dell'ultimo miglio), una delle fasi più critiche e costose della catena di distribuzione moderna. In particolare, il dominio di riferimento è quello delle spedizioni Amazon, caratterizzato da un elevato volume di ordini giornalieri, vincoli temporali stringenti e una forte dipendenza da fattori ambientali dinamici.

Le principali sfide affrontate in questo dominio includono:

- **Incertezza Ambientale:** Il tempo di consegna è influenzato da variabili stocastiche come le condizioni meteorologiche (pioggia, nebbia, tempesta) e la densità del traffico urbano.
- **Vincoli Operativi:** Una delle fasi critiche riguarda l'assegnazione di un set di ordini ai corrieri disponibili. Tale processo deve soddisfare vincoli rigidi di capacità dei veicoli (differenziati tra motocicli e furgoni) e vincoli di competenza relativi alla qualità del servizio e alla tipologia di merce trattata.
- **Ottimizzazione Spaziale:** Il problema è stato modellato come una ricerca del cammino minimo su un grafo stradale complesso. L'obiettivo è l'identificazione della rotta ottimale che congiunge il punto di prelievo **(Store)** alla destinazione finale **(Drop-off)**, minimizzando i costi di percorrenza attraverso algoritmi di ricerca informata.

1.2 Knowledge Based System (KBS)

La Knowledge Base (KB) rappresenta l'insieme delle informazioni strutturate necessarie al funzionamento del sistema. Partendo dal dataset iniziale, i dati vengono elaborati per alimentare correttamente i processi di ricerca, ottimizzazione e ragionamento logico.

1.3 Origine dei Dati

La fonte primaria di conoscenza è il dataset pubblico *"Amazon Delivery Dataset"* (disponibile su Kaggle). Il file originale, `amazon_delivery.csv`, contiene circa 45.000 record storici caratterizzati da attributi eterogenei, tra cui:

- **Dati Contestuali:** Condizioni meteo (es. Stormy, Sunny) e traffico (es. Jam, Low).
- **Dati Anagrafici:** Età e rating dei corrieri.
- **Dati Geografici:** Coordinate GPS di partenza e arrivo.

1.4 Costruzione della Conoscenza (Preprocessing)

I dati grezzi presentavano rumore e inconsistenze che ne impedivano l'utilizzo diretto. È stata implementata una pipeline di preprocessing per generare il file pulito `amazon_delivery_final.csv`, necessario per massimizzare l'accuratezza dei modelli predittivi. Le operazioni principali hanno incluso:

1. **Data Cleaning:** Rimozione di record con coordinate GPS errate (valori 0.0 o latitudini negative) e gestione degli outlier tramite metodo IQR.
2. **Feature Engineering:** Calcolo della **Distanza di Haversine** a partire dalle coordinate grezze, creando una nuova metrica fondamentale per la stima dei tempi.
3. **Trasformazione dei Dati:**
   - **Standardizzazione:** Applicazione di `StandardScaler` sulle variabili numeriche (Età, Rating) per normalizzare la distribuzione (media 0, deviazione standard 1).
   - **One-Hot Encoding:** Conversione delle variabili categoriche (Meteo, Traffico) in vettori binari (dummy variables) per evitare l'introduzione di ordinamenti fittizi tipici del Label Encoding.

Questa fase ha permesso di validare le relazioni tra le variabili (tramite analisi di correlazione di Spearman), eliminando feature ridondanti e fornendo una base di conoscenza robusta per gli esperimenti successivi.

1.4.1 Analisi delle Correlazioni

A supporto delle decisioni di preprocessing, è stata condotta un'analisi delle correlazioni di Spearman per validare le relazioni tra le variabili. Come mostrato nella **Figura 1**, è possibile osservare le interdipendenze tra le feature principali, mentre la **Figura 2** evidenzia l'impatto specifico delle variabili sul target `Delivery_Time` (es. correlazione positiva per il traffico e negativa per il rating).

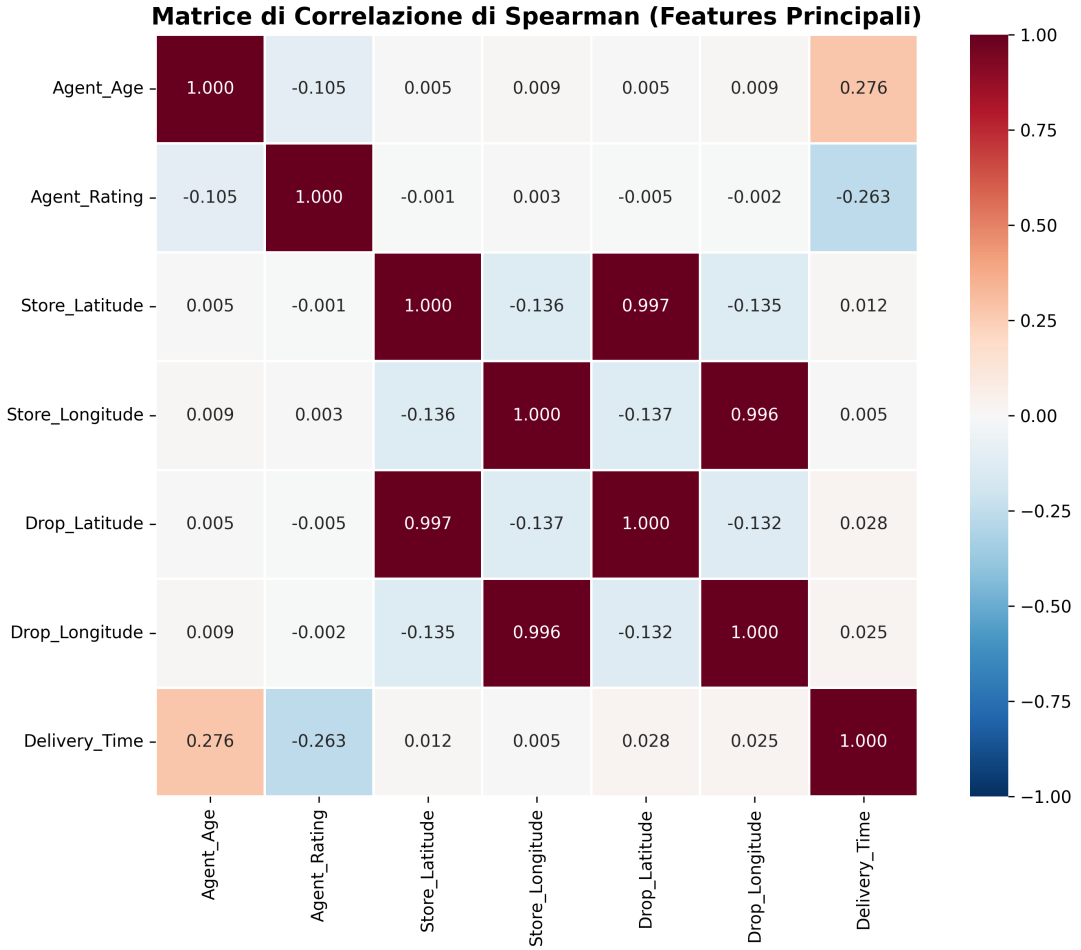

Figura 1: **Heatmap di Spearman:** Matrice di correlazione tra le features numeriche principali.

- **Meteo e Traffico vs Tempi (Correlazione Positiva)**: Le condizioni avverse, come il maltempo (es. `Weather_Stormy`) e gli ingorghi stradali (`Traffic_Jam`), mostrano una chiara correlazione positiva con i tempi di consegna (`Delivery_Time`).

  Impatto: All'aumentare della criticità ambientale, i tempi operativi crescono fisiologicamente a causa della riduzione della velocità media e della visibilità.

- **Performance Agente vs Tempi (Correlazione Negativa)**: Esiste una relazione inversamente proporzionale tra il punteggio di valutazione del fattorino (`Agent_Rating`) e i tempi di consegna.

  Impatto: Un *rating* più elevato è statisticamente associato a una maggiore efficienza operativa, risultando in una riduzione dei tempi di completamento della missione.

- **Età Agente (Correlazione Debole)**: La variabile `Agent_Age` influisce solo marginalmente sui tempi operativi o sulla probabilità di utilizzare un determinato tipo di veicolo.

  Impatto: L'esperienza legata all'età non costituisce un fattore discriminante primario per la velocità di consegna nel contesto urbano analizzato.

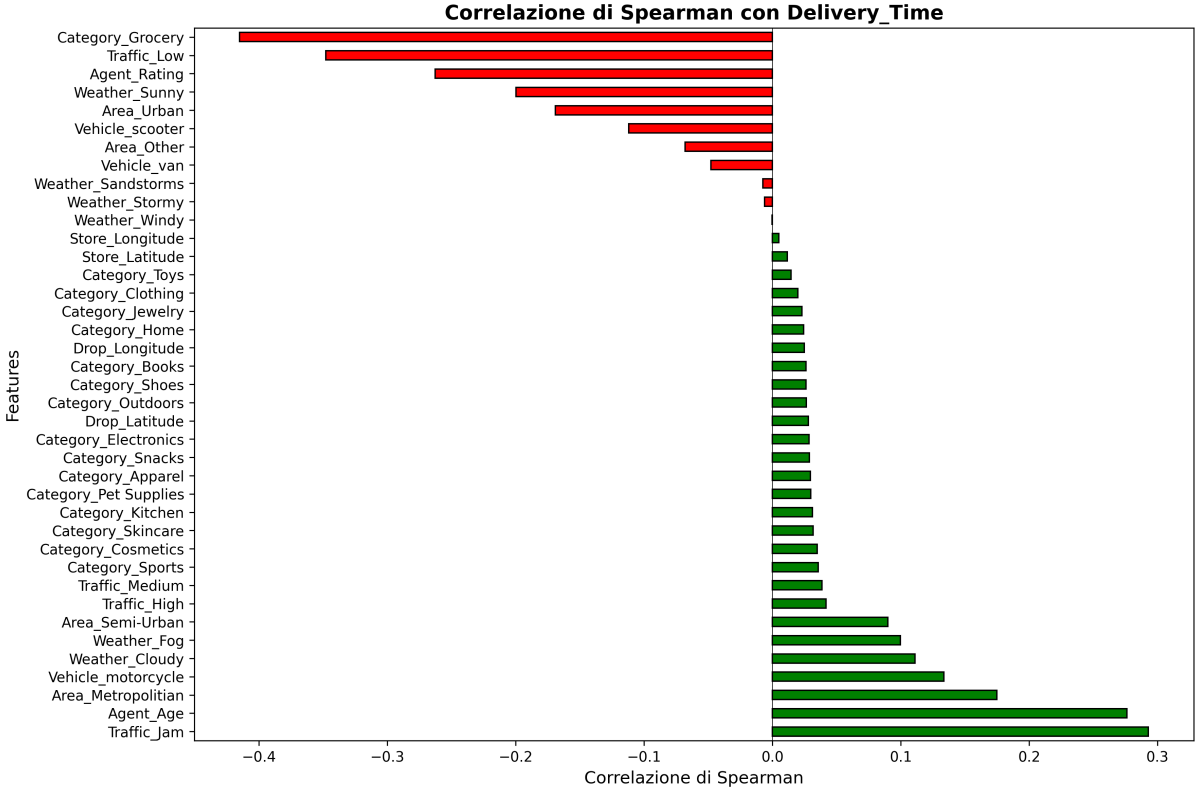

Figura 2: **Analisi Target:** Correlazione delle variabili rispetto al tempo di consegna. Le barre verdi indicano fattori che aumentano il tempo (es. Traffic Jam), quelle rosse fattori che lo riducono.

2 Apprendimento Supervisionato

2.1 Sommario

Il modulo di Machine Learning costituisce la componente predittiva del sistema e affronta un problema di **Regressione**, finalizzato a stimare con precisione i tempi di consegna per gli ordini Amazon. Il sistema elabora come input il dataset pre-elaborato (`amazon_delivery_final.csv`), il quale organizza in una matrice di caratteristiche (X) variabili quali il meteo, il traffico e altri fattori logistici. Il risultato finale di questa elaborazione consiste nella generazione del file `amazon_delivery_with_prediction.csv`, contenente i tempi di consegna previsti dal modello. L'architettura è stata addestrata con l'obiettivo di minimizzare l'errore quadratico medio (**MSE**) e massimizzare il coefficiente di determinazione (R^2), garantendo così che l'intero KBS si basi su stime temporali estremamente affidabili.

2.2 Strumenti utilizzati

La realizzazione del modulo si avvale dello stack scientifico Python standard:

- **Pandas & NumPy:** Per il caricamento e la manipolazione algebrica dei vettori strutturati.
- **Scikit-learn:** Nucleo algoritmico per l'implementazione dei modelli (`LinearRegression`, `RandomForestRegressor`), il partizionamento dei dati (`train_test_split`) e la validazione incrociata.
- **XGBoost:** Libreria ottimizzata per algoritmi di Gradient Boosting, utilizzata per confronto.
- **Joblib:** Per la serializzazione (persistenza) del modello addestrato.

2.3 Decisioni di Progetto

2.3.1 Trattamento dei dati

Il dataset è stato suddiviso staticamente in un **Training Set (80%)** e un **Test Set (20%)**, mantenendo un `random_state` fisso per la riproducibilità. Per il modello (Regressione Lineare), è stata applicata la standardizzazione tramite `StandardScaler`, mentre per gli algoritmi basati su alberi si è operato sui dati grezzi, sfruttando la loro invarianza di scala.

2.3.2 Selezione dell'Algoritmo

La selezione del modello è stata guidata da un approccio comparativo incrementale, valutando il trade-off tra complessità e capacità di generalizzazione:

1. **Baseline (Regressione Lineare):** I risultati hanno evidenziato un *underfitting*, confermando la natura non lineare delle relazioni tra variabili (es. interazione traffico-meteo) e l'incapacità del modello di catturare pattern complessi.

2. **Decision Tree:** L'uso di un albero singolo senza vincoli di profondità ha mostrato un forte *overfitting* (alta varianza). Nonostante le ottime performance sul training set, il modello ha mostrato una scarsa capacità di generalizzare su dati non visti.

3. **XGBoost (Extreme Gradient Boosting):** Testato come modello allo stato dell'arte basato sulla tecnica del **Boosting**. Sebbene abbia ottenuto lo score R^2 più elevato, è stato mantenuto come termine di paragone (*benchmark*) a causa della sua elevata sensibilità agli iperparametri e del rischio di adattarsi eccessivamente al rumore presente nel dataset di consegna.

4. **Random Forest (Scelta Finale):** Si è optato per il `RandomForestRegressor` (con 100 stimatori). Questo metodo di *Ensemble Learning* riduce la varianza tramite la tecnica del **Bagging** (Bootstrap Aggregating). Mediando le predizioni di molteplici alberi decorrelati, il modello garantisce una maggiore robustezza agli outlier e una stabilità superiore, requisiti fondamentali per informare i successivi moduli decisionali (CSP).

2.3.3 Validazione della Robustezza (K-Fold Cross Validation)

Per garantire che le performance del modello *Random Forest* non fossero dipendenti da una specifica suddivisione favorevole del dataset, è stata eseguita una **5-Fold Cross Validation**. Questo processo ha permesso di valutare la capacità di generalizzazione dell'agente su cinque diverse partizioni dei dati di training.

I risultati ottenuti, riportati nella Tabella 1, confermano l'elevata affidabilità del sistema:

Fold	R^2 **Score**
Fold 1	0.7266
Fold 2	0.7257
Fold 3	0.7415
Fold 4	0.7234
Fold 5	0.7283
Media (R^2)	**0.7291**
Deviazione Standard (σ)	**0.0064**

Tabella 1: Risultati della 5-Fold Cross Validation per il modello Random Forest

Analisi dei risultati: L'elemento di maggior rilievo è rappresentato dalla **Deviazione Standard ($\sigma = 0.0064$)**, estremamente contenuta. Tale valore indica che le oscillazioni di performance tra i diversi fold sono trascurabili, dimostrando che il modello ha una varianza molto bassa e una grande stabilità. La coerenza tra il punteggio medio della cross-validation (0.7291) e il risultato ottenuto sul test set conferma ulteriormente che il modello non è affetto da *overfitting* e fornisce una base di conoscenza solida per i successivi moduli di ragionamento probabilistico e ottimizzazione.

2.4 Valutazione

La valutazione è stata condotta utilizzando le metriche standard MSE, RMSE e R^2. I risultati comparativi sono riassunti nella Tabella 2 e visualizzati graficamente nella Figura 3.

Modello	R^2 **Score**	**RMSE**	**Esito**
Linear Regression	0.61	High	Scartato (Underfitting)
Decision Tree	$\sim$ 0.51	High	Scartato (Overfitting)
Random Forest	**0.73**	**Low**	**Selezionato**
XGBoost	0.76	Lowest	Benchmark

Tabella 2: Confronto numerico delle performance sul Test Set

Il confronto evidenzia che i modelli ensemble (Random Forest e XGBoost) superano nettamente la regressione lineare, riducendo sensibilmente l'errore di predizione ($RMSE$). In questo contesto, XGBoost è stato utilizzato esclusivamente come benchmark per attestare la conoscenza di soluzioni allo stato dell'arte. La scelta finale è tuttavia ricaduta sul Random Forest poiché garantisce il miglior equilibrio tra precisione e stabilità operativa, assicurando al contempo coerenza con il programma didattico e i modelli approfonditi durante il corso.

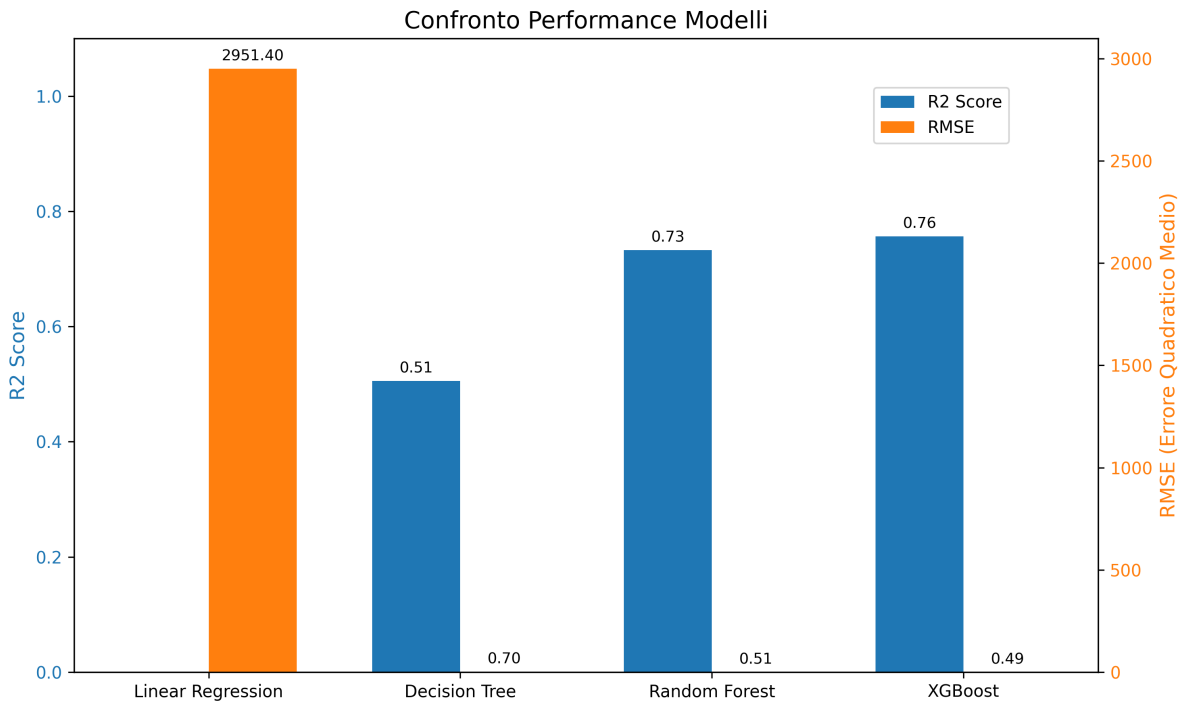

Figura 3: **Confronto Modelli:** Si nota il netto divario prestazionale. Il Random Forest offre un ottimo compromesso tra accuratezza ($R^2 > 0.7$) e stabilità, avvicinandosi molto al benchmark XGBoost.

2.4.1 Analisi dei Risultati (Random Forest)

Per validare ulteriormente il modello selezionato, è stata effettuata un'analisi dei residui visibile in Figura 4. L'allineamento dei punti lungo la diagonale (linea rossa tratteggiata) conferma che le predizioni sono coerenti con i valori reali, mantenendo una varianza contenuta anche per tempi di consegna elevati.

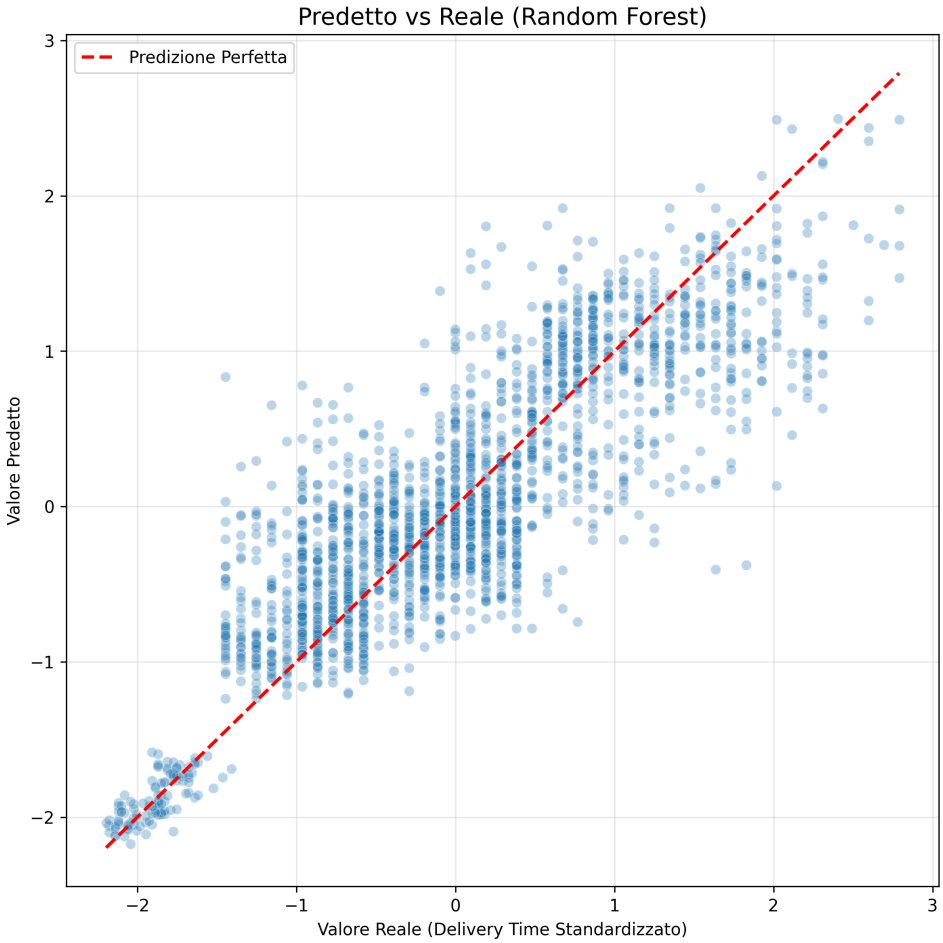

Figura 4: **Predetto vs Reale:** Scatter plot delle predizioni. La concentrazione dei punti lungo la bisettrice indica un'elevata capacità di generalizzazione del modello.

Infine, per garantire l'interpretabilità del modello, è stata estratta l'importanza delle feature (Figura 5). È interessante notare come la tipologia di ordine (`Category_Grocery`) e l'affidabilità del corriere (`Agent_Rating`) giochino un ruolo predominante rispetto alla pura distanza geografica, suggerendo che fattori logistici e umani sono critici quanto quelli spaziali.

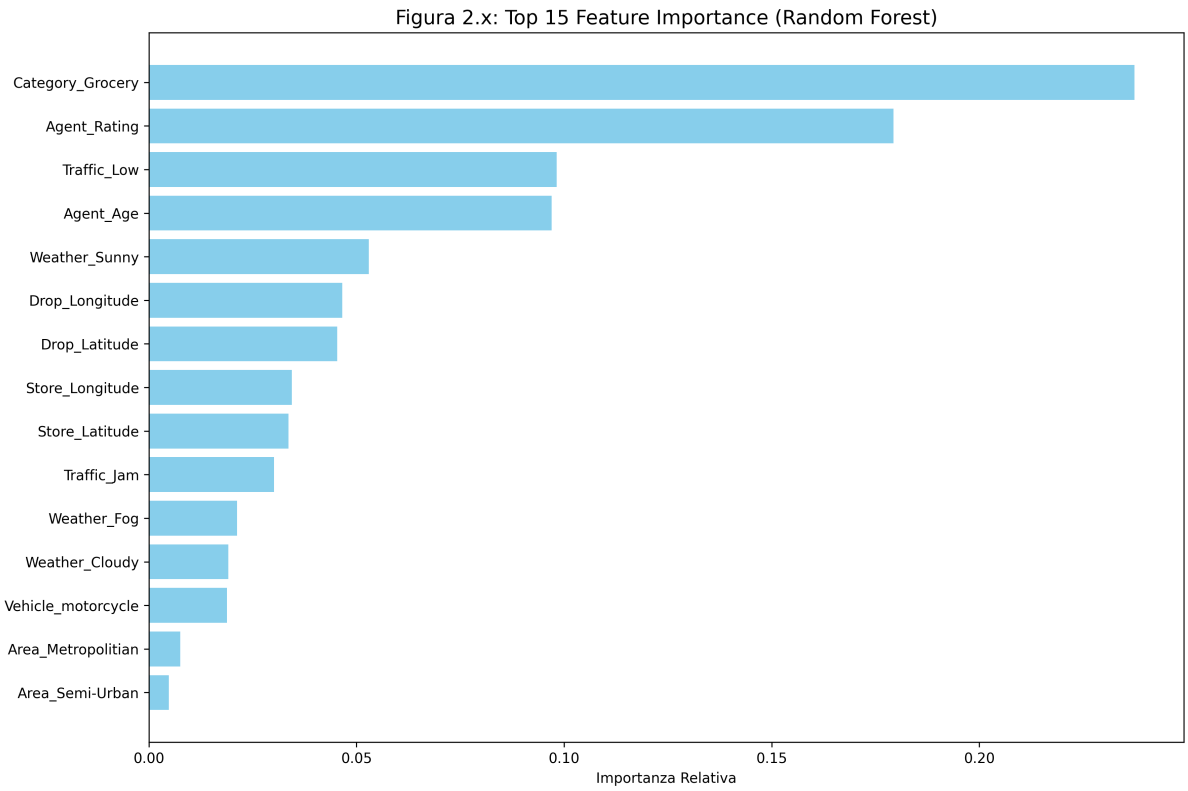

Figura 5: **Feature Importance:** I fattori che più influenzano la predizione. Si evidenzia il peso critico della categoria merceologica e del rating del fattorino.

3 Ragionamento con Vincoli

3.1 Sommario

Questo modulo affronta una delle fasi più complesse della pianificazione logistica: l'assegnazione ottimale di un set di ordini ai corrieri disponibili. Tale operazione viene modellata come un Problema di Soddisfacimento di Vincoli (CSP - Constraint Satisfaction Problem). L'obiettivo del sistema non è la semplice distribuzione del carico, ma l'individuazione di una configurazione (assegnazione) che garantisca la massima efficienza operativa senza violare i vincoli imposti dal dominio. Il modulo riceve in input la lista degli ordini e l'elenco dei corrieri attivi, cercando una soluzione in cui ogni ordine sia associato al profilo di agente e al mezzo di trasporto più idoneo.

3.2 Strumenti utilizzati

- **Pandas & NumPy**: Manipolazione dati, filtraggio geografico di Mumbai e normalizzazione dei rating degli agenti.
- **Random & Time**: Gestione della stocasticità negli algoritmi di ricerca e monitoraggio delle performance temporali.
- **Matplotlib**: Visualizzazione grafica della convergenza e della distribuzione dei risultati delle assegnazioni.
- **Collections**: Analisi statistica per la verifica del corretto bilanciamento del carico di lavoro tra i corrieri.

3.3 Decisioni di Progetto

Le decisioni di progetto si sono concentrate sulla creazione di uno scenario logistico che fosse al contempo realistico e bilanciato. Si è scelto di analizzare esclusivamente l'area di **Mumbai** per la sua elevata densità urbana, fattore che permette di testare i vincoli del sistema in un contesto operativo complesso.
La flotta è stata configurata includendo **sei furgoni** e **sei motocicli**, così da valutare la capacità del risolutore di gestire mezzi con diverse disponibilità di carico. Infine, il problema è stato dimensionato su un set di **30 ordini**, una misura ideale per garantire risultati significativi senza rallentare eccessivamente i tempi di calcolo degli algoritmi di ricerca e ottimizzazione.

3.3.1 Trattamento dei Dati

Il trattamento dei dati ha permesso di trasformare il dataset grezzo in una base di conoscenza strutturata per il modulo CSP attraverso i seguenti passaggi:

- **Filtro Geografico**: È stata isolata l'area metropolitana di Mumbai tramite coordinate spaziali. Questo assicura che il sistema operi su un raggio d'azione coerente, evitando assegnazioni tra punti eccessivamente distanti.
- **De-standardizzazione dei Rating**: I punteggi degli agenti sono stati riportati alla scala originale 1.0 - 5.0. Questa operazione di *Knowledge Normalization* è

fondamentale per permettere ai vincoli logici di operare su soglie di merito reali e facilmente interpretabili.

- **Dataset di Output**: Il risultato finale è la creazione del file `csp_mumbai_espanso.csv`, dove i dati vengono organizzati distinguendo chiaramente tra variabili (ordini) e domini (agenti), pronti per l'elaborazione algoritmica.

3.3.2 Modellazione del Problema (CSP)

Il problema è stato formalizzato come un CSP (*Constraint Satisfaction Problem*) dove l'obiettivo è l'assegnazione ottimale delle risorse:

- **Variabili (X)**: I 30 ordini di consegna estratti nell'area di Mumbai.
- **Domini (D)**: I 12 agenti disponibili (6 dotati di Van e 6 di Motociclo).
- **Vincoli (C)**:
  - *Capacità Veicolo (Hard)*: Limite massimo di 5 ordini per i Van e 2 per i Motocicli.
  - *Capacità Specializzata (Hard)*: Per la categoria `Home` (pesanti), limite di 3 pacchi per i Van e 1 per i Motocicli.
  - *Coerenza Territoriale (Hard)*: L'agente deve operare esclusivamente nell'area geografica (Urban, Metropolitan, Semi-Urban) del primo ordine assegnato.
  - *Requisito Qualità (Hard)*: Gli ordini `Electronics` possono essere gestiti solo da agenti con un Rating superiore a 3.8.

3.4 Scelta dell' Algoritmo

In questo modulo sono stati implementati e confrontati tre diversi approcci per la risoluzione del problema di assegnazione: una ricerca sistematica (**Backtracking**) e due approcci di ricerca locale (**Simulated Annealing** e **Iterative Best Improvement**). L'obiettivo è stato individuare l'algoritmo capace di offrire il miglior **compromesso tra velocità di esecuzione e qualità della soluzione**.

3.4.1 Backtracking Search

Il **Backtracking** è stato scelto come algoritmo di riferimento (*baseline*) per la sua capacità di esplorazione sistematica e completa. In un contesto di logistica con vincoli rigidi (capacità e area), un algoritmo che garantisce di trovare una soluzione esatta rappresenta il punto di partenza ideale per validare la modellazione del problema.

- **Logica di Funzionamento**: L'algoritmo opera secondo una strategia di ricerca in profondità (*DFS - Depth-First Search*). Esso tenta di assegnare un agente a ogni ordine in modo sequenziale: prima di confermare ogni assegnazione parziale, il motore verifica il rispetto dei quattro vincoli definiti. Se un'assegnazione viola anche un solo parametro (es. l'incoerenza dell'area geografica tra ordine e agente), l'algoritmo esegue un "passo indietro" (*backtrack*), annullando l'ultima scelta e tentando con un agente differente.

- **Limiti riscontrati (Timeout)**: Nonostante la dimensione contenuta del problema (30 ordini), la fase di test ha evidenziato un fallimento dell'algoritmo per *timeout*. Questo è dovuto alla natura sovra-vincolata (*overconstrained*) del dataset di Mumbai: l'estrema rigidità dei vincoli geografici e di capacità genera uno spazio di ricerca talmente vasto che la ricerca sistematica non riesce a convergere verso una soluzione ammissibile in tempi utili.

- **Valore Diagnostico**: Sebbene non sia stato in grado di produrre un'assegnazione finale, l'impiego del *Backtracking* è stato fondamentale a livello progettuale. Ha infatti permesso di dimostrare matematicamente l'impossibilità di soddisfare contemporaneamente tutti i vincoli operativi con le risorse a disposizione, evidenziando la necessità di passare ad approcci di ricerca locale più flessibili e capaci di gestire soluzioni sub-ottime.

3.4.2 Simulated Annealing (SA)

Il **Simulated Annealing** è stato implementato come alternativa di ricerca locale stocastica. A differenza del Backtracking, che costruisce una soluzione passo dopo passo, il SA parte da un'assegnazione completa (seppur casuale e potenzialmente ricca di conflitti) e tenta di migliorarla iterativamente attraverso un processo ispirato al raffreddamento dei metalli in metallurgia.

- **Logica di Funzionamento**: L'algoritmo definisce una funzione di che corrisponde al numero totale di violazioni dei vincoli presenti nell'assegnazione. Ad ogni iterazione, viene proposta una modifica casuale (spostamento di un ordine a un nuovo agente). Se la modifica riduce il numero di conflitti, viene accettata immediatamente. Se invece peggiora la soluzione, può comunque essere accettata con una certa probabilità, regolata da un parametro chiamato "Temperatura" (T).

- **Perché è stato scelto**: Questo approccio è stato introdotto per la sua capacità di sfuggire ai *minimi locali*. Mentre algoritmi più semplici (come l'*Greedy Ascent*) potrebbero bloccarsi in una configurazione non ottimale ma difficile da migliorare, il SA permette di accettare temporaneamente mosse peggiorative per esplorare nuove aree dello spazio degli stati, tendendo verso l'ottimo globale con il diminuire della temperatura.

- **Parametri di Progetto**: Il sistema è stato configurato con una temperatura iniziale elevata ($T = 10$) e un tasso di raffreddamento geometrico (`cooling_rate` $= 0.99$). Questo schema permette una fase iniziale di esplorazione caotica utile a diversificare le soluzioni, seguita da una fase in cui l'algoritmo si stabilizza sulla migliore configurazione trovata.

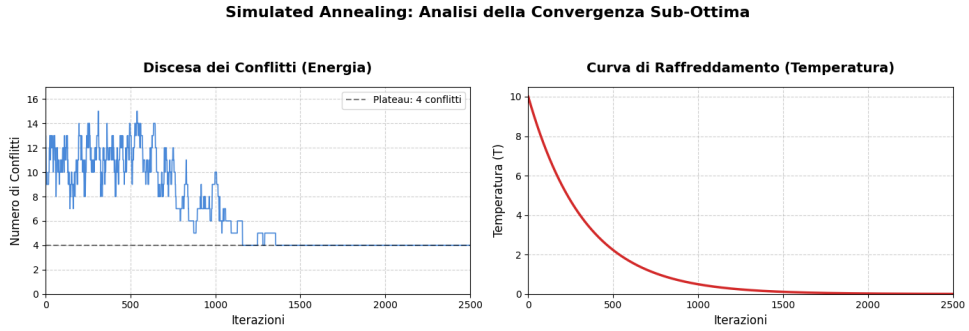

Figura 6: Rappresentazione Simulated Annealing.

3.4.3 Iterative Best Improvement

L'ultimo approccio testato appartenente agli algoritmi di ricerca è IBI. A differenza del *Simulated Annealing*, che può accettare mosse peggiorative per esplorare lo spazio degli stati, questo algoritmo segue una logica più diretta e focalizzata alla risoluzione immediata dei conflitti.

- **Logica di Funzionamento**: L'algoritmo inizia con un'assegnazione completa di tutti i 30 ordini, generata in modo casuale e quindi inizialmente ricca di violazioni. Il motore identifica ciclicamente quali ordini sono in conflitto (ovvero quali violano i vincoli di capacità, area o rating) e tenta di trovare la miglior configurazione che soddisfi i vincoli. Per ogni ordine critico, viene valutato l'impatto di tutti i possibili agenti del dominio e viene scelto quello che minimizza il numero totale di conflitti nel sistema.

- **Perché è stato scelto**: È stato incluso nel progetto per la sua straordinaria efficienza nel risolvere problemi *CSP* densi. Mentre il *Backtracking* costruisce la soluzione, l'*Iterative Best Improvement* la corregge iterativamente, rendendolo spesso molto più rapido nel gestire scenari dove una soluzione quasi corretta è già disponibile o facile da raggiungere.

- **Strategia di Evasione**: Per evitare che l'algoritmo rimanga bloccato in cicli infiniti tra due configurazioni simili, la scelta della variabile da riparare avviene in modo stocastico tra tutte quelle che presentano violazioni.

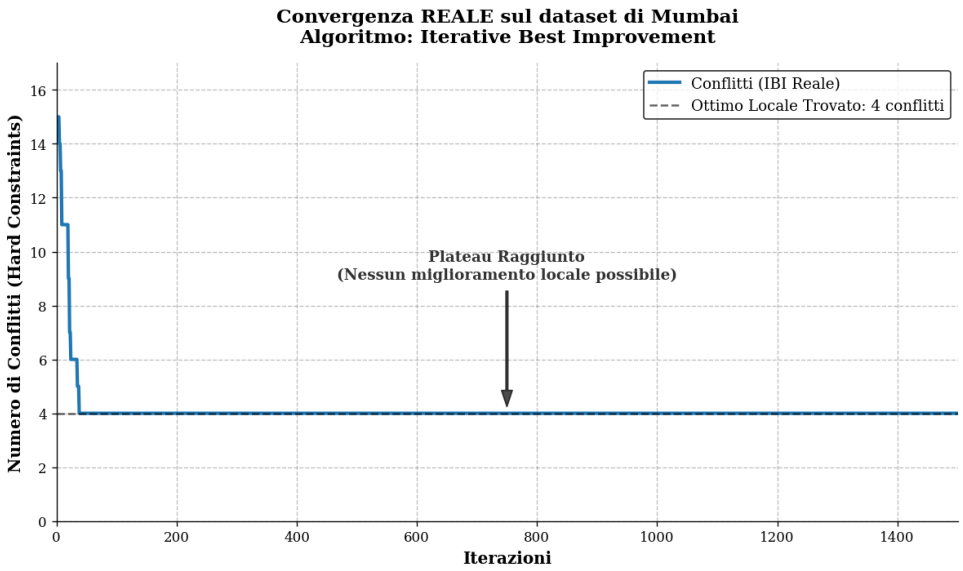

Figura 7: Rappresentazione Iterative Best Improvement.

3.5 Valutazione e Risultati

In questa fase finale, i tre algoritmi sono stati messi a confronto diretto sullo stesso scenario operativo (30 ordini e 12 agenti). L'obiettivo della valutazione è misurare la capacità dei motori di ricerca di gestire vincoli geografici e di capacità in un contesto ad alta densità. L'analisi dei dati evidenzia una natura sovra-vincolata (*overconstrained*) del problema proposto. Di seguito le osservazioni tecniche principali:

- **Il fallimento del Backtracking**: La ricerca sistematica è andata in *timeout* dopo aver esplorato oltre 270.000 nodi. Questo accade perché il *Backtracking*, per sua natura, non accetta compromessi: se la combinazione perfetta non esiste, l'algoritmo continuerà a tentare milioni di combinazioni (effetto *thrashing*). La rigidità dei vincoli sulle aree di Mumbai ha reso impossibile un'assegnazione perfetta.
- **L'efficacia delle Ricerche Locali**: Sia il *Simulated Annealing* che il *Iterative Best-Improvements* hanno fornito una risposta in tempi brevissimi (meno di 0.06s). Sebbene non abbiano trovato una soluzione con pochi conflitti, di fatto si sono stabilizzati su una soluzione sub-ottima con soli 4 conflitti residui. In un contesto logistico reale, questo è un risultato accettabile per garantire l'operatività.
- **Confronto SA vs IBI**: Il *Simulated Annealing* si è dimostrato leggermente più rapido nel processare le iterazioni, mentre il *Iterative Best-Improvements* ha raggiunto lo stesso livello di errore con meno passaggi, confermando la sua validità nel riparare velocemente assegnazioni parzialmente corrette.

3.6 Conclusioni sul Motore Scelto

Nonostante la precisione teorica del *Backtracking*, per la gestione operativa su larga scala la scelta ricade sugli algoritmi di **Ricerca Locale**. Questi ultimi garantiscono la continuità del servizio anche quando le risorse non sono sufficienti a soddisfare tutti i vincoli

Algoritmo	**Stato Soluzione**	**Conflitti**	**Nodi / Iter.**	**Tempo (s)**
Backtracking (DFS)	Fallito (Timeout)	N/A	271.298	10.00
Simulated Annealing	Parziale	4	2.500	0.012
Iterative Best-Improvement	Parziale	4	1.000	0.078

Tabella 3: Tabella Comparativa delle Prestazioni

contemporaneamente, permettendo al gestore logistico di intervenire manualmente solo sulle poche criticità residue.

4 Ricerca di Soluzioni in Spazi di Stati

4.1 Sommario

L'obiettivo di questa sezione è trovare il percorso a costo minimo che colleghi il punto di ritiro (*Store*) al punto di consegna (*Drop*), attingendo ai dati del dataset preprocessato `amazon_delivery_final.csv`. In stretta coerenza con il modulo precedente, le operazioni sono state filtrate per isolare esclusivamente l'area di **Mumbai**, garantendo così una continuità territoriale nell'analisi. Il problema viene modellato come una ricerca in uno spazio degli stati rappresentato da un **grafo sintetico**, dove ogni nodo identifica una posizione e ogni arco una via di comunicazione. A differenza di una semplice navigazione basata sulla distanza, questo modulo introduce la complessità del mondo reale integrando il **fattore traffico**. Il costo di percorrenza di una strada non dipende quindi solo dalla sua lunghezza fisica, ma viene pesato dinamicamente in base alle condizioni di viabilità rilevate nel dataset (da *Low* a *Jam*).

4.2 Strumenti utilizzati

- **OSMnx**: È stata utilizzata per scaricare, modellare e visualizzare il grafo stradale reale dell'area di Mumbai direttamente da *OpenStreetMap*. Permette di trasformare le infrastrutture urbane in una struttura dati a grafo compatibile con gli algoritmi di ricerca.
- **NetworkX**: Questa libreria è servita per la gestione interna delle strutture dei grafi, consentendo la manipolazione dei nodi (incroci) e degli archi (strade) e l'accesso alle proprietà fisiche del percorso, come la lunghezza dei segmenti.
- **Folium**: Impiegata per la visualizzazione cartografica interattiva. È stata fondamentale per proiettare su mappa i risultati degli algoritmi, distinguendo graficamente tra il percorso ottimale trovato e l'area dello spazio degli stati effettivamente esplorata.
- **Pandas**: Utilizzata per il caricamento del dataset preprocessato `amazon_delivery_final.csv` e per le operazioni di filtraggio spaziale tramite le coordinate di latitudine e longitudine.
- **Heapq**: Fondamentale per l'efficienza degli algoritmi di ricerca (A*, *Greedy*, *LCFS*). È stata utilizzata per implementare la coda di priorità (frontiera), garantendo che l'estrazione del nodo successivo avvenga sempre con complessità logaritmica.
- **Math (Haversine)**: Le funzioni trigonometriche della libreria `math` sono state utilizzate per implementare la formula di *Haversine*, necessaria per calcolare la distanza in linea d'aria tra due punti terrestri, fungendo da base per la funzione euristica.

4.3 Scelta Algoritmi

4.3.1 Algoritmo di Ricerca A*

L'algoritmo **A*** è stato scelto come motore principale per la navigazione grazie alla sua capacità di combinare l'efficienza della ricerca euristica con la garanzia di ottimalità tipica degli algoritmi sistematici.

- **Funzione di Valutazione**: L'algoritmo opera minimizzando la funzione $f(n) = g(n) + h(n)$, dove:
  - $g(n)$: Rappresenta il costo effettivo (lunghezza della strada in metri) percorso dal punto di partenza allo stato attuale.
  - $h(n)$: È l'euristica che stima il costo rimanente per raggiungere la destinazione.
- **Euristica Geografica**: Per il calcolo di $h(n)$ è stata implementata la **formula di Haversine**, che determina la distanza in linea d'aria tra le coordinate GPS del nodo corrente e quelle del punto di consegna (*Drop*). Tale euristica è considerata *ammissibile*, poiché la distanza in linea d'aria non sovrastima mai la distanza stradale effettiva, garantendo così che il percorso trovato sia sempre il più breve possibile.
- **Gestione della Ricerca**: L'implementazione utilizza una coda di priorità (`heapq`) per gestire la frontiera dei nodi da esplorare. Ad ogni passo, il sistema estrae il nodo con il valore di $f(n)$ minore, riducendo drasticamente lo spazio di ricerca rispetto a un algoritmo non informato (come *LCFS*), pur mantenendo la stessa precisione.
- **Tracking dell'Esplorazione**: Il codice include una funzione *custom* per tracciare l'intero spazio degli stati visitato durante la ricerca. Questa operazione permette di visualizzare graficamente quanto l'algoritmo sia stato diretto nel raggiungere l'obiettivo rispetto ai nodi totali del grafo stradale di Mumbai.

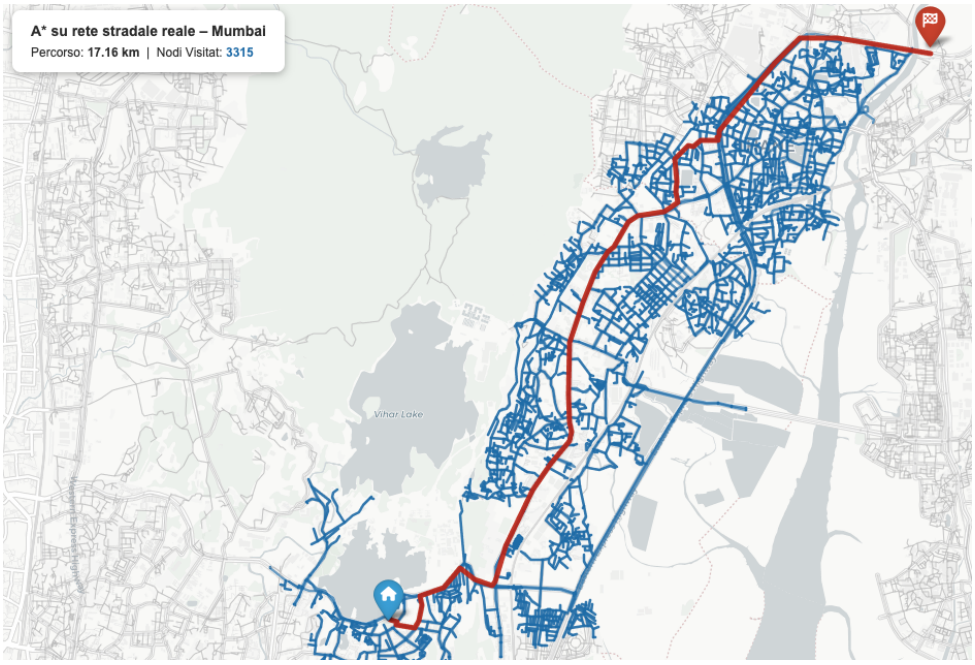

Figura 8: Visualizzazione dell'espansione dell'algoritmo A*.

4.3.2 Greedy Best-First Search

L'algoritmo **Greedy Best-First Search** è stato implementato come secondo termine di paragone per valutare l'impatto delle euristiche pure sulla velocità di ricerca, a scapito dell'ottimalità del percorso.

- **Funzione di Valutazione**: A differenza di A^*, il *Greedy* opera ignorando completamente il costo accumulato dal punto di partenza ($g(n)$). La sua funzione di valutazione è ridotta alla sola componente euristica: $f(n) = h(n)$.

- **Logica "Ingorda"**: L'algoritmo è definito "ingordo" poiché, ad ogni passo, sceglie il nodo che sembra più vicino alla meta in linea d'aria, senza considerare se la strada percorsa finora sia stata efficiente. Questo lo porta a dirigersi in modo estremamente aggressivo verso l'obiettivo.

- **Euristica e Performance**: Utilizzando la **formula di Haversine** come euristica, il *Greedy* riduce drasticamente il numero di nodi esplorati rispetto a una ricerca sistematica. Tuttavia, non garantendo l'ottimalità, il percorso finale può risultare sensibilmente più lungo rispetto a quello calcolato da A^*, poiché l'algoritmo potrebbe bloccarsi davanti a ostacoli geografici (es. vicoli ciechi o deviazioni stradali) che lo costringono a percorsi tortuosi.

- **Visualizzazione nel Sistema**: Nell'output cartografico generato, l'area esplorata dal *Greedy* viene evidenziata in **verde**, mentre il percorso finale è tracciato in **arancione**. Questa distinzione permette di osservare visivamente come lo spazio di ricerca del *Greedy* sia molto più stretto e focalizzato rispetto agli altri approcci.

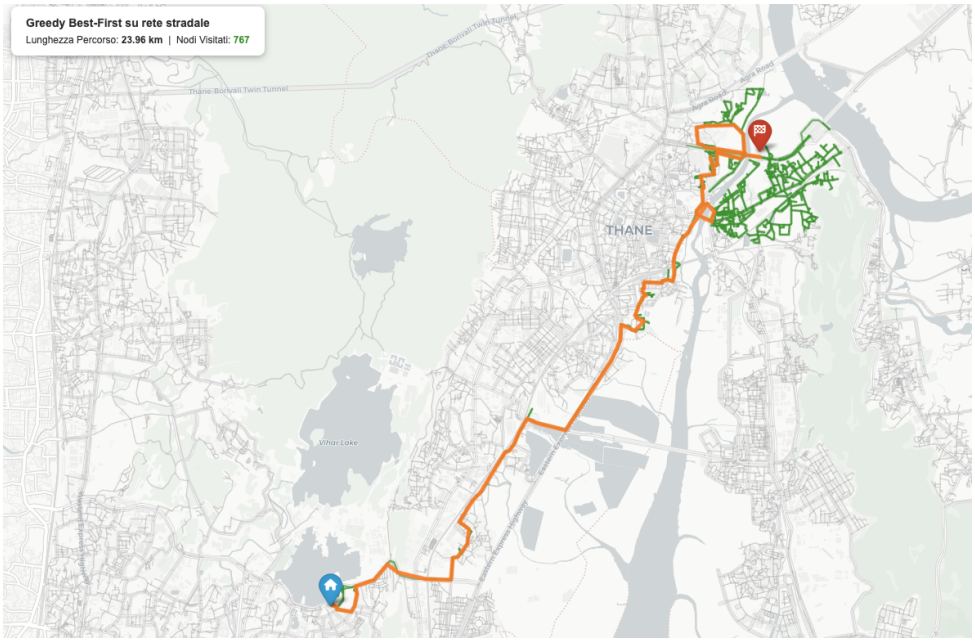

Figura 9: Visualizzazione dell'espansione dell'algoritmo Greedy.

s

4.3.3 Lowest Cost First Search

L'algoritmo **lowest Cost First Search** (noto anche come algoritmo di *Dijkstra*) è stato implementato come termine di paragone per la ricerca sistematica non informata, rappresentando il limite massimo di esplorazione del sistema.

- **Funzione di Valutazione**: A differenza degli algoritmi euristici, il *LCFS* basa le proprie decisioni esclusivamente sul costo accumulato: $f(n) = g(n)$. Non avendo

alcuna informazione sulla direzione della meta, l'algoritmo espande i nodi in modo uniforme in tutte le direzioni.

- **Espansione "a macchia d'olio"**: Proprio per la mancanza di un'euristica ($h(n) = 0$), il comportamento del *LCFS* è visivamente paragonabile a una macchia d'olio che si espande dal punto di partenza. L'algoritmo esplora ogni possibile strada finché non incrocia casualmente il nodo di destinazione, garantendo comunque l'ottimalità del percorso trovato ma con uno sforzo computazionale elevato.

- **Efficienza e Nodi Espansi**: Nel *benchmark* del sistema, il *LCFS* funge da "limite superiore" per il numero di nodi espansi. Poiché deve analizzare ogni incrocio prima di scartarlo, il numero di nodi visitati è sensibilmente maggiore rispetto ad A^* e *Greedy*, evidenziando l'importanza di una ricerca informata in contesti urbani densi come Mumbai.

- **Visualizzazione nel Sistema**: Sulla mappa interattiva generata tramite `Folium`, l'area esplorata dal *LCFS* viene colorata in **viola**, mentre il percorso finale è tracciato in **rosso scuro**. Questo permette di apprezzare visivamente la differenza tra una ricerca che "conosce" la meta e una che esplora l'ambiente in modo puramente basato sulla distanza percorsa.

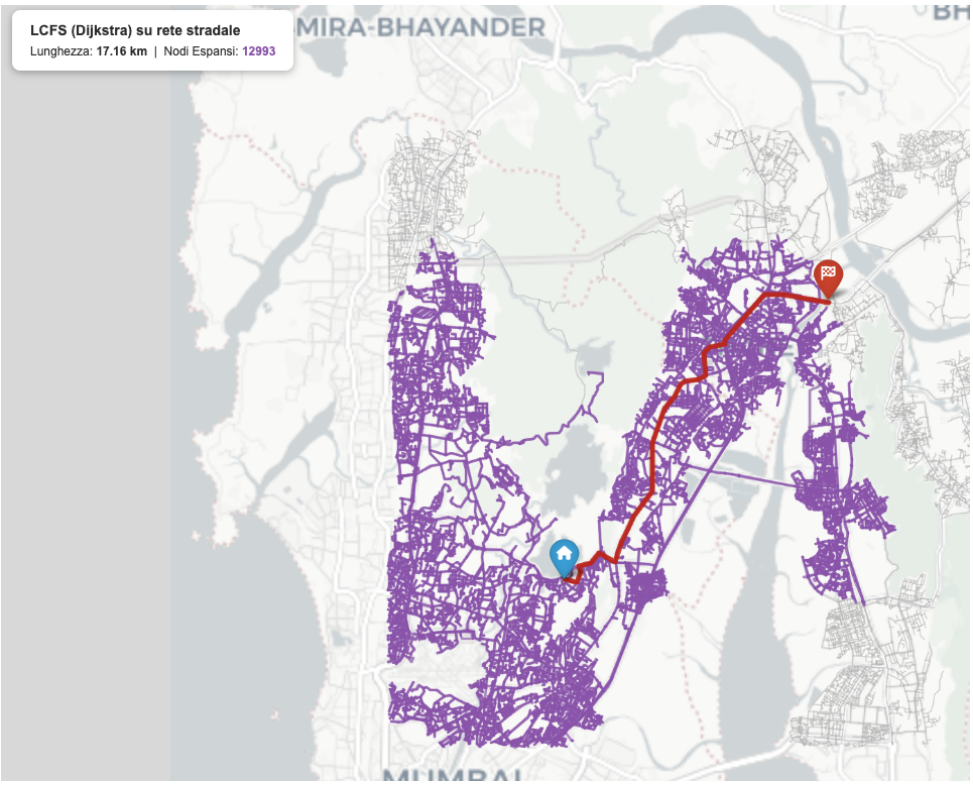

Figura 10: Visualizzazione dell'espansione dell'algoritmo LCFS.

4.4 Valutazione

Questa sezione finale del modulo di navigazione mette a confronto le prestazioni reali dei tre algoritmi testati sulla rete stradale di Mumbai. I test sono stati effettuati su un

tragitto di circa 13.77 km (in linea d'aria), permettendo di osservare come l'uso delle euristiche influenzi sia la qualità del percorso che lo sforzo computazionale. Per garantire l'imparzialità del test, tutti i motori di ricerca hanno operato sullo stesso grafo stradale composto da **20.634 nodi** e **47.196 archi**.

Algoritmo	**Ottimalità**	**Distanza (km)**	**Nodi Espansi**	**Efficienza**
LCFS (Dijkstra)	**Ottimo**	17.16 km	12.993	Bassa
Greedy Best-First	Sub-ottimo	23.96 km	767	Altissima
A*	**Ottimo**	**17.16 km**	**3.315**	**Bilanciata**

Tabella 4: Confronto delle prestazioni reali sulla rete stradale di Mumbai.

4.4.1 Analisi dell'Efficienza (Nodi Espansi)

Il dato più significativo emerge dal confronto tra *LCFS* e A^*. Entrambi gli algoritmi hanno trovato il percorso matematicamente più breve (17.16 km), ma con uno sforzo radicalmente diverso:

- **LCFS**: operando senza alcuna guida euristica ($f(n) = g(n)$), ha dovuto espandere quasi il 63% del grafo (12.993 nodi) prima di raggiungere la meta.
- **A***: grazie alla funzione euristica $h(n)$ basata sulla formula di *Haversine*, ha ridotto l'esplorazione a soli 3.315 nodi.

Insight: L'uso dell'informazione geografica ha permesso ad A^* di risparmiare circa il 75% di risorse computazionali rispetto a *LCFS*, pur garantendo lo stesso identico risultato.

4.4.2 Analisi della Qualità (Ottimalità del Percorso)

Il limite della ricerca puramente euristica emerge chiaramente nell'analisi del *Greedy Best-First Search*:

- Sebbene sia stato l'algoritmo più veloce in termini di esplorazione (solo 767 nodi visitati), ha prodotto un percorso di 23.96 km. Questo risultato è circa il 40% più lungo rispetto al percorso ottimo trovato da A^*.
- **Il motivo**: Ignorando il costo del cammino già percorso ($g(n)$), il *Greedy* prosegue scegliendo strade che sembravano dirette ma che in realtà allungavano drasticamente il tragitto effettivo.

4.4.3 Conclusione Algoritmo Scelto : A*

A* si rivela il motore ideale per la navigazione urbana a Mumbai: garantisce l'**ottimalità dei percorsi** (superando l'imprecisione del *Greedy*) con un'efficienza computazionale nettamente superiore al *LCFS*. È l'unica soluzione capace di conciliare **risparmio di carburante** e **velocità operativa**.

5 Ragionamento Probabilistico (Bayesian Network)

5.1 Sommario

L'obiettivo di questo modulo è modellare le dipendenze probabilistiche tra le variabili ambientali e l'esito operativo delle consegne, introducendo nel sistema la capacità di ragionare in **condizioni di incertezza**. A differenza dei moduli precedenti, che operano su logiche deterministiche (percorsi ottimi o soddisfacimento di vincoli rigidi), la **Rete Bayesiana** permette di gestire la natura stocastica del mondo reale. Utilizzando il dataset preprocessato `dataset_bayes_mumbai.csv` e filtrato per l'area di Mumbai, il modello non si limita a predire un valore esatto, ma calcola la **probabilità condizionata** di un evento — nello specifico il rischio di ritardo (*Delay_Risk*) — basandosi sull'osservazione di evidenze quali le condizioni meteorologiche (*Weather*), l'intensità del traffico (*Traffic*) e la tipologia di zona (*Area*). Questo approccio permette al sistema di effettuare sia **previsioni dirette** (*stress test*) che **diagnostica inversa**, risalendo alle cause più probabili di un fallimento logistico.

5.2 Strumenti Utilizzati

- **pgmpy**: Rappresenta il cuore tecnologico del modulo. Questa libreria specializzata è stata utilizzata per definire la struttura del grafo aciclico diretto (*DAG*) tramite la classe `DiscreteBayesianNetwork`, per l'apprendimento dei parametri e per l'esecuzione del motore di inferenza.

- **BayesianEstimator**: Questo specifico stimatore di `pgmpy` è stato preferito ai metodi classici di massima verosimiglianza. È servito per calcolare le tabelle di probabilità condizionata (*CPT*), permettendo l'integrazione di una conoscenza a priori (*BDeu prior*) che garantisce la robustezza del modello anche in presenza di combinazioni di dati rare nel dataset.

- **VariableElimination**: È l'algoritmo di inferenza esatta impiegato per interrogare la rete. Attraverso questo strumento è stato possibile calcolare la distribuzione di probabilità marginale della variabile `Delay_Risk` una volta osservate le evidenze (*meteo, traffico, area*).

- **Pandas e NumPy**: Queste librerie sono state fondamentali nella fase di *data engineering*. `Pandas` è servita per il filtraggio dei record nell'area di Mumbai e la pulizia delle categorie, mentre `NumPy` è stato utilizzato per la discretizzazione del *target*, trasformando il tempo di consegna continuo nella variabile binaria di rischio.

5.3 Decisioni di Progetto

La progettazione della Rete Bayesiana ha richiesto scelte strategiche per trasformare i dati grezzi in un modello di ragionamento capace di gestire l'incertezza tipica dell'ambiente urbano di *Mumbai*.

5.3.1 Architettura del Grafo e Dipendenze Causali

La decisione principale ha riguardato la definizione del **DAG** (*Directed Acyclic Graph*). Si è scelto di implementare una struttura a connessione convergente (*v-structure*), dove più

variabili indipendenti confluiscono verso un unico nodo "effetto". Questa scelta permette di modellare come diverse cause ambientali collaborino nel determinare il rischio finale. Visualmente, la gerarchia delle dipendenze è così strutturata:

- *Weather* → `Delay_Risk`: Influenza diretta dovuta alla riduzione della velocità e della visibilità.
- *Traffic* → `Delay_Risk`: Influenza primaria legata alla saturazione delle vie stradali.
- *Area* → `Delay_Risk`: Influenza strutturale basata sulla complessità urbanistica della zona.

Questa configurazione abilita il fenomeno dell' *Explaining Away*: se osserviamo un ritardo (`Delay_Risk=High`) e sappiamo che il traffico era scarso (*Traffic=Low*), la rete aumenterà automaticamente la probabilità che la causa sia stata il maltempo, fornendo un ragionamento diagnostico estremamente realistico.

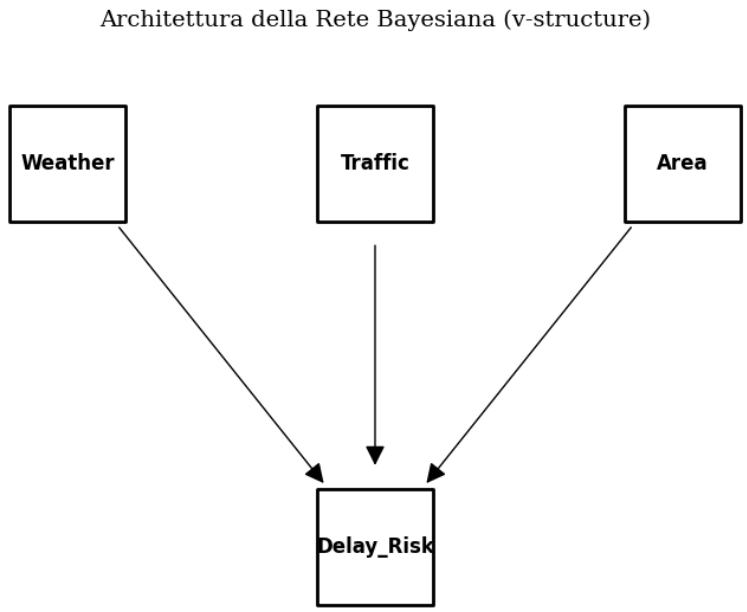

Figura 11: Visualizzazione della struttura della rete.

5.3.2 Discretizzazione delle Variabili

Per rendere computazionalmente trattabile il calcolo delle tabelle **CPT**, i dati continui e le stringhe sono stati mappati in stati discreti:

- ***Weather***: Suddiviso in 6 stati (*Cloudy, Fog, Sandstorms, Stormy, Sunny, Windy*) per catturare ogni sfumatura climatica di Mumbai.
- ***Traffic***: Ridotto a 4 livelli critici (*Low, Medium, High, Jam*).
- ***Area***: Mappata su 4 categorie spaziali (*Urban, Metropolitan, Semi-Urban, Other*).
- ***Delay_Risk* (Target)**: È stata operata una scelta binaria strategica. Basandosi sulla distribuzione dei tempi di consegna nel dataset, è stata fissata una soglia di 150 minuti:

- *Low*: Consegna entro i tempi standard.
- *High*: Rischio critico di ritardo (superamento della soglia dei 150 min).

5.3.3 Strategia di Apprendimento e Gestione dell'Incertezza

Una decisione fondamentale ha riguardato l'uso del **Bayesian Estimator** con *BDeu prior* (*Bayesian Dirichlet equivalent uniform*) al posto del classico *MLE* (*Maximum Likelihood Estimation*).

- **Il problema**: In un dataset reale, alcune combinazioni sono rarissime (es. *Sandstorm + Urban + Traffic Low*). Un approccio standard assegnerebbe probabilità zero a questi eventi, "rompendo" il modello in caso di tali evidenze.
- **La soluzione**: L'estimatore Bayesiano introduce una conoscenza a priori, garantendo che il sistema sia sempre in grado di fornire una risposta probabilistica (anche se minima) per qualsiasi scenario, rendendo l'IA robusta e resiliente ai dati mancanti.

5.4 Sviluppo del Modello: Apprendimento e Inferenza

Una volta definita la struttura e preparati i dati, il modulo è stato reso operativo attraverso due fasi sequenziali: l'apprendimento dei parametri e la configurazione del motore di calcolo probabilistico.

5.4.1 Apprendimento delle Tabelle di Probabilità Condizionata (CPT)

Il cuore del modello risiede nelle tabelle **CPT** (*Conditional Probability Tables*), che quantificano numericamente le relazioni tra i nodi. In questa fase:

- Il dataset `dataset_bayes_mumbai.csv` è stato processato dall'algoritmo *Bayesian Estimator*.
- Il sistema ha analizzato i **3.037 record** validi nell'area di Mumbai per calcolare come la distribuzione del rischio di ritardo cambi al variare di ogni combinazione di traffico, meteo e area.
- Grazie alla *BDeu prior* con una dimensione del campione equivalente (*ESS - Equivalent Sample Size*) pari a 10, il modello ha ammorbidito le frequenze statistiche: questo garantisce che anche per gli scenari meno frequenti (es. meteo *Fog* in area *Urban*), la rete generi una distribuzione di probabilità coerente anziché basarsi su dati isolati o nulli.

5.4.2 Configurazione del Motore di Inferenza

Per estrarre conoscenza dalla rete, è stato implementato un motore basato sull'algoritmo di **Eliminazione delle Variabili** (*Variable Elimination*). Questa tecnica di inferenza esatta permette di marginalizzare le variabili non rilevanti e concentrare il calcolo solo sulla *query* di interesse. Il sistema è stato impostato per operare in due modalità:

- **Inferenza Predittiva (*Forward Reasoning*)**: Il sistema riceve evidenze sulle cause (es. *Traffic=Jam*) e calcola la distribuzione di probabilità sull'effetto (`Delay_Risk`). È la modalità utilizzata per prevedere se una consegna pianificata sia a rischio prima ancora che il corriere parta.
- **Inferenza Diagnostica (*Backward Reasoning*)**: Il sistema riceve l'evidenza dell'effetto (es. ritardo avvenuto) e risale alle probabilità delle cause. Questa funzione è fondamentale per l'analisi post-operativa, permettendo di capire se i fallimenti logistici siano stati causati da una gestione errata dei flussi o da eventi meteorologici imprevedibili.

5.5 Valutazione dei Risultati

La valutazione è stata condotta interrogando il motore di inferenza su scenari critici e osservazioni reali, con l'obiettivo di validare la coerenza del modello rispetto alle dinamiche urbane.

5.5.1 Analisi degli Scenari

Attraverso lo stress test degli scenari di rischio, abbiamo calcolato la probabilità che il rischio di ritardo sia elevato (P(`Delay_Risk` = High | Evidenza)). I risultati mostrano una chiara correlazione tra le variabili ambientali e l'affidabilità della consegna:

Profilo Scenario	**Evidenza (Meteo + Traffico)**	**Rischio di Ritardo (High)**
Critico 1	*Jam + Cloudy*	**68.6%**
Critico 2	*Jam + Stormy*	**36.5%**
Ottimale	*Low + Sunny*	13.7%
Area Metropolitan	— (Strutturale)	29.9%
Area Urban	— (Strutturale)	18.2%

Tabella 5: Stress test degli scenari di rischio predetto.

Analisi del Rischio: I dati evidenziano come la congestione stradale (*Jam*) sia il moltiplicatore di rischio più potente. È interessante notare come la combinazione *Jam + Cloudy* raggiunga quasi il 70% di probabilità di ritardo, superando persino lo scenario *Stormy*. Questo suggerisce che, nel contesto di *Mumbai*, le giornate nuvolose siano statisticamente associate a colli di bottiglia logistici più severi, probabilmente a causa di un volume di traffico superiore rispetto alle giornate di tempesta, dove la mobilità generale potrebbe risultare ridotta.

5.5.2 Diagnostica Inversa (Backward Reasoning)

Per testare la capacità del modello di risalire alle determinanti di un disservizio, abbiamo ipotizzato un evento di ritardo critico avvenuto (`Delay_Risk` = High) per risalire alla causa primaria. L'inferenza ha prodotto la seguente distribuzione probabilistica sulla variabile *Traffico*:

- P(Traffico = Jam | Ritardo): **49.9% (Causa Primaria)**

- $P(\text{Traffico} = \text{Medium} \mid \text{Ritardo})$: 27.5%
- $P(\text{Traffico} = \text{High} \mid \text{Ritardo})$: 11.4%
- $P(\text{Traffico} = \text{Low} \mid \text{Ritardo})$: 11.2%

Il modello identifica correttamente il traffico di tipo "Jam" come il responsabile principale di circa la metà dei ritardi critici, validando la sensibilità della rete alle dinamiche di saturazione urbana.

6 Ragionamento Proposizionale (Prolog)

6.1 Sommario

L'obiettivo di questo modulo è la creazione di un **Sistema Esperto** capace di automatizzare le scelte pratiche e le regole di gestione dell'azienda. Mentre i moduli precedenti si occupano di aspetti come l'ottimizzazione dei percorsi o la stima probabilistica del rischio, **Prolog** agisce come la parte decisionale del sistema. Invece di limitarsi a elaborare dati, questo modulo li trasforma in azioni concrete seguendo le linee guida operative. Il sistema attinge direttamente dal dataset iniziale `amazon_delivery_start.csv` — ovvero il dataset non processato — estraendo fatti riguardanti gli agenti, le condizioni del traffico e le caratteristiche degli ordini. Usando queste informazioni la sua **Base di Conoscenza** permette di:

- **Assegnare premi in modo oggettivo**: Identificare i corrieri che meritano un bonus basandosi sulla qualità del loro lavoro (*rating*) e sulla zona in cui operano (*area*).
- **Attivare alert per i clienti**: Generare avvisi automatici quando la combinazione di maltempo e traffico rende probabile un ritardo, aumentando la trasparenza.
- **Gestire le priorità di consegna**: Decidere quali pacchi (come i prodotti di elettronica) devono essere consegnati con precedenza assoluta.

6.2 Strumenti Utilizzati

- **Pandas**: Utilizzata per caricare il dataset grezzo (`amazon_delivery_start.csv`) ed estrarre le informazioni necessarie (*ID, rating, meteo, traffico*).
- **Random**: Necessaria per selezionare casualmente un campione di agenti e ordini dal dataset, così da generare una base di fatti diversificata per il test della logica.
- **OS**: Impiegata per la gestione dei percorsi dei file nel sistema e per la creazione fisica del file della *Knowledge Base* (`.pl`).
- **SWI-Prolog (Sintassi)**: Rappresenta il linguaggio in cui viene generato l'output. Definisce le regole di *business* e gestisce l'inferenza per derivare bonus e *alert*.

6.3 Decisioni di Progetto

La progettazione della *Knowledge Base* e delle regole logiche è stata guidata dall'obiettivo di creare un sistema trasparente e facilmente manutenibile. Per lo sviluppo di questo modulo, sono state adottate le seguenti scelte strategiche:

6.3.1 Campionamento dei dati per la verifica deduttiva

Si è optato per l'estrazione di un campione mirato di 5 record dal dataset originale non processato (`amazon_delivery_start.csv`).

- **Motivazione**: A differenza dei modelli di *Machine Learning*, **Prolog** non richiede una fase di addestramento su grandi moli di dati, poiché opera per inferenza deduttiva su fatti certi. Questa scelta permette di garantire la totale tracciabilità del ragionamento logico.

6.3.2 Definizione dei criteri di merito

È stata fissata la soglia per l'assegnazione del bonus a un Rating ≥ 4.7, limitatamente agli agenti operanti nell'area *Metropolitan*.

- **Motivazione**: Tale scelta è volta a premiare esclusivamente la fascia più alta dei collaboratori (*top performers*), garantendo un incentivo legato a standard di servizio superiori alla media.

6.3.3 Categorizzazione delle priorità e mitigazione del rischio

È stato stabilito che gli ordini appartenenti alla categoria *Electronics* ricevano automaticamente una priorità alta.

- **Motivazione**: Questa regola risponde a una logica di gestione del rischio e del valore. Trattandosi di beni caratterizzati da alta fragilità e valore economico elevato, è prioritario minimizzare il loro tempo di permanenza nel ciclo di consegna, riducendo così l'esposizione a potenziali danneggiamenti o furti.

6.3.4 Architettura modulare e scalabilità

Il codice è stato strutturato separando i fatti base dai predicati ausiliari (come `meteo_critico` e `traffico_critico`).

- **Motivazione**: Questa impostazione segue i principi di modularità del software. Isolare la definizione di ciò che è "critico" dalle regole di *business* vere e proprie rende il sistema estremamente flessibile: se le *policy* aziendali dovessero cambiare, è possibile aggiornare i criteri di rischio modificando una singola riga di codice, senza dover intervenire sulla struttura logica complessiva.

6.4 Modellazione Logica

La *Base di Conoscenza* (KB) è il cuore del Sistema Esperto e si articola in due componenti fondamentali: i fatti, che rappresentano lo stato attuale del mondo logistico a Mumbai, e le regole, che definiscono le relazioni logiche necessarie per l'automazione decisionale.

6.4.1 Rappresentazione dei Fatti (Knowledge Base)

I fatti vengono generati dinamicamente tramite lo script di estrazione. La modellazione prevede due predicati principali che mappano i dati grezzi in relazioni logiche:

- `agente(ID, Rating, Area)`: Caratterizza i corrieri.
  - *Esempio*: `agente(agente_2, 4.8, 'Metropolitian')` indica un operatore con alta valutazione in una zona ad alta densità.
- `ordine(ID, Meteo, Traffico, Categoria)`: Descrive le condizioni operative della singola consegna.
  - *Esempio*: `ordine(xqmf197034060, 'Windy', 'Medium', 'Electronics')`.

6.4.2 Logica delle Regole di Business

Le regole sono state scritte utilizzando la sintassi delle **clausole di Horn** per permettere al motore di inferenza di derivare conclusioni attraverso la risoluzione *SLD*.

6.4.3 Regola del Bonus Merito

La logica implementata premia l'efficienza in condizioni di alta complessità urbana:

$$assegna_bonus(A) \leftarrow agente(A, R, 'Metropolitian') \wedge R \geq 4.7$$

L'agente ottiene il bonus se e solo se opera nell'area *Metropolitian* e mantiene un *rating* di eccellenza.

6.4.4 Regola di Alert Logistico (Notifica Ritardo)

Questa regola utilizza i predicati ausiliari (`meteo_critico` e `traffico_critico`) per identificare situazioni di rischio combinato:

$$notifica_ritardo(O) \leftarrow ordine(O, W, T, _) \wedge meteo_critico(W) \wedge traffico_critico(T)$$

Un *alert* viene generato solo quando si sovrappongono un evento meteorologico avverso e una congestione stradale elevata.

6.4.5 Regola di Gestione Priorità

Filtra gli ordini basandosi sulla criticità intrinseca della categoria merceologica:

$$priorita_alta(O) \leftarrow ordine(O, _, _, 'Electronics')$$

Questa regola garantisce che i beni di alto valore o fragilità siano processati con precedenza dal sistema decisionale.

6.4.6 Implementazione delle Regole Decisionali

Di seguito viene riportata la codifica in linguaggio **Prolog** delle politiche decisionali implementate. La struttura utilizza la logica dichiarativa per trasformare i fatti estratti dal *dataset* in direttive operative per la gestione della logistica a *Mumbai*.

```
% 1. POLITICA DI INCENTIVI (Meritocrazia)
% Un agente ottiene il bonus se mantiene un'eccellenza nel rating
% e opera nella zona piu' complessa (Metropolitian).
assegna_bonus(A) :-
    agente(A, R, 'Metropolitian'),
    R >= 4.7.

% 2. GESTIONE ALERT E RISCHIO (Sicurezza Operativa)
% Genera una notifica di ritardo solo in presenza di "Doppio
   Rischio":
% sovrapposizione di meteo avverso e traffico critico.
notifica_ritardo(O) :-
    ordine(O, W, T, _),
    meteo_critico(W),
```

```
    traffico_critico(T).

% 3. CATEGORIZZAZIONE DELLE PRIORITA (Valore e Fragilita')
% Assegna priorita' alta ai prodotti elettronici per ridurre
% l'esposizione al rischio durante il trasporto.
priorita_alta(O) :-
    ordine(O, _, _, 'Electronics').

% 4. DEFINIZIONE DEL DOMINIO CRITICO (Fatti Ausiliari)
meteo_critico('Stormy').
meteo_critico('Fog').
meteo_critico('Sandstorms').

traffico_critico('Jam').
traffico_critico('High').
```

Listing 1: Regole decisionali in Prolog per la logistica di Mumbai

6.5 Valutazione

La validazione del sistema esperto è stata effettuata interrogando la *Knowledge Base* attraverso *query* specifiche. I risultati, riassunti nelle tabelle seguenti, confermano la corretta applicazione della logica deduttiva sui fatti estratti dal *dataset*.

6.5.1 Valutazione Bonus Merito (Agenti)

La *policy* aziendale prevede l'assegnazione del bonus esclusivamente per *Rating* ≥ 4.7 e Area *'Metropolitian'*. Il motore di inferenza ha prodotto i seguenti esiti:

Agente	**Rating**	**Area**	**Esito Bonus**	**Motivazione Logica**
agente_1	4.7	Urban	Negativo	Area non idonea
agente_2	4.8	Metropolitian	**POSITIVO**	Target raggiunto
agente_3	4.8	Urban	Negativo	Area non idonea
agente_4	4.9	Metropolitian	**POSITIVO**	Target raggiunto
agente_5	4.6	Urban	Negativo	Rating e Area insufficienti

Tabella 6: Validazione della regola `assegna_bonus/1`.

6.5.2 Valutazione Alert e Priorità (Ordini)

L'*alert* di ritardo viene attivato in presenza di meteo e traffico critici, mentre la priorità alta è riservata alla categoria *'Electronics'*.

ID Ordine	Meteo / Traffico	Categoria	Esito Logico	Analisi
hroe078...	Cloudy / Medium	Pet Supplies	Standard	Condizioni entro i limiti
cjhk414...	Sandstorms / Jam	Snacks	**ALERT RITARDO**	Doppio rischio critico
qkux956...	Stormy / Medium	Grocery	Standard	Traffico non critico
xqmf197...	Windy / Medium	Electronics	**PRIORITÀ ALTA**	Merce fragile/valore

Tabella 7: Validazione delle regole `notifica_ritardo/1` e `priorita_alta/1`.

6.6 Conclusioni sul Modulo Logico

I test dimostrano che il sistema basato su **Prolog** è in grado di operare come un supervisore intelligente. A differenza di un semplice filtraggio dati, la capacità di concatenare fatti ausiliari (come la definizione di criticità) permette di implementare politiche aziendali complesse in modo dichiarativo. Il sistema si è dimostrato robusto, trasparente nel ragionamento e facilmente scalabile per l'aggiunta di nuovi protocolli operativi.

7 Conclusioni e Sviluppi Futuri

A conclusione di questo lavoro, sono stati individuati due percorsi principali per evolvere ulteriormente il sistema, con l'obiettivo di renderlo ancora più efficiente e vicino alle necessità operative di una città complessa come *Mumbai*.

7.1 Integrazione e Sinergia tra i Moduli

Attualmente, i cinque moduli (*Apprendimento Supervisionato ,CSP, Ricerca in Spazi di Stati, Rete Bayesiana e Ragionamento Proposizionale*) operano in modo indipendente, ognuno risolvendo una parte specifica del problema. Uno sviluppo futuro naturale consiste nel creare una comunicazione diretta e automatica tra di essi.

- **Scenario Operativo**: Se il modulo Bayesiano rileva un aumento improvviso del rischio di ritardo su una rotta, questa informazione potrebbe essere inviata immediatamente al modulo *CSP*. Il sistema potrebbe quindi decidere in tempo reale di riassegnare l'ordine a un altro corriere più vicino o con un carico minore, evitando che il ritardo si accumuli.

In questo modo, l'intero ecosistema IA passerebbe da una serie di passaggi isolati a un **ciclo decisionale continuo** e fluido.

7.2 Utilizzo di Flussi di Dati in Tempo Reale

Il progetto attuale si basa sull'analisi di un *dataset* storico. Per rendere il sistema davvero reattivo, il passo successivo sarebbe l'integrazione di dati in tempo reale tramite **API esterne** (come *Google Maps* per il traffico o servizi meteo *live*).

- **Ottimizzazione Dinamica**: Invece di calcolare il percorso con A^* su dati passati, il sistema potrebbe aggiornare la rotta dell'autista ogni 5 minuti basandosi su un incidente stradale appena avvenuto.
- **Reattività Logica**: Il modulo *Prolog* potrebbe inviare un *alert* "Notifica Ritardo" basandosi sulla pioggia rilevata in quel preciso istante, migliorando drasticamente la precisione delle comunicazioni al cliente e la gestione delle emergenze.

7.3 Chiusura del Progetto

In definitiva, l'architettura proposta dimostra come la combinazione di diverse tecniche di Intelligenza Artificiale — dalla ricerca euristica alla logica simbolica e probabilistica — possa fornire una soluzione olistica a problemi logistici complessi. L'implementazione su *Mumbai* funge da *proof-of-concept* per un sistema scalabile, trasparente e pronto per le sfide della moderna *Smart City*.

Riferimenti Bibliografici

Materiale Didattico e Dataset

[1] Dispense del corso di *Ingegneria della Conoscenza (ICON)*, Università degli Studi di Bari Aldo Moro, Dipartimento di Informatica, A.A. 2025/2026.

[2] F. A. Anwar, "Amazon Delivery Dataset," *Kaggle*. [Online]. Available: https://www.kaggle.com/code/fareedalianwar/amazon-delivery.

Software e Librerie Scientifiche

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[4] T. Chen e C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[5] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126-139, 2017.

[6] A. Ankan e A. Panda, "pgmpy: A python library for learning and inference in Bayesian Networks," in *Proceedings of the 14th Python in Science Conference (SCIPY)*, 2015.

[7] J. Wielemaker, T. Schrijvers, M. Triska, e T. Lager, "SWI-Prolog," *Theory and Practice of Logic Programming*, vol. 12, no. 1-2, pp. 67-96, 2012.

[8] A. Hagberg, D. Schult, e P. Swart, "NetworkX: Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy)*, 2008.

Documentazione Tecnica

[9] Documentazione ufficiale *Folium*, libreria per la visualizzazione di dati geospaziali su mappe interattive.

[10] Documentazione *Pandas* e *NumPy*, librerie per la manipolazione algebrica e strutturale dei dati in Python.